

New Paltz
STATE UNIVERSITY OF NEW YORK

Modified Black Jack Game

Tanvir Khan

State University of New York at New Paltz

EGC320-01-Digital System Design

Professor Radhakrishnan

18 December 2019

Abstract:

Created a modified black jack game that would only have two players that uses a random generator, addition, seven-segments displays, switches, and sequential circuits. Player that has the greater number wins. This game allows for two people to play. Many modifications were made to improve the aesthetics of the game.

Table of Contents

Introduction	3
Theory	3
Design	4
Random Generator	4
Seven-segment display	5
Addition	6
Switches and physical setup	6
Results and Conclusion	8
References	9
Appendices	10
Appendix A: System.v	10
Appendix B: bcdAdder2.v	20
Appendix C: Binary2BCD.v	22
Appendix D: bcdAdder.v	23
Appendix E: random6.v	25
Appendix F: bcd_to_seven_seg2.v	27
Appendix G: bcd_to_seven_seg.v	31
Appendix H: random2	32
Appendix I: random3	34
Appendix I: random2	37
Appendix J: Pin Assignments	40

Introduction

The purpose of this project was to use a game that uses sequential circuits. Additionally sequential circuits, such as connected D-flip-flops. The project would have two players playing a game to see which player would go be close to 21 first. There is only one card, the player has to pick up. Meaning there is only one number that gets randomly generated that can be generated. The closer the player score is greater than the other player score, which gives that player a 100% chance to win the game.

Theory

This project was broken down into four main parts: random number generator, addition, seven-segment-display, and the rules of the game. Each part can be related to a specific purpose in hardware generally. Since a random generator is needed, research needed to be done on linear feed-back shift registers. Since verilog is a hardware based code, there is no such thing is random, but pseudo-random. Pseudo-random is a sequence of digits that has a pattern. The linear feedback shift register is used as the Pseudo-random algorithm. In the linear feedback shift register there are D-flip flops used and the last two bit are exclusively ORed together.

The addition needs to be done in bcd, which is binary coded decimal. Binary coded decimal takes four bits of binary code at a time. For example, the binary code with 8 bits of 10000010, translates to 82.

The seven-segment display is useful in showing numbers from 0 to 9 and letters from A to F. The seven-segment display is used for communicating letters or numbers on an fpga board. The seven-segment display takes zero as power and one is ground. Meaning that giving a zero to a segment lights it up, otherwise giving a one does not light it up. The letters are in a,b,c,d,e,f,g. These letters correspond to seven segments. In the DE2-115 manual, the letters are represented by numbers. The letters correspond to 0,1,2,3,4,5,6. These figures can be seen below.

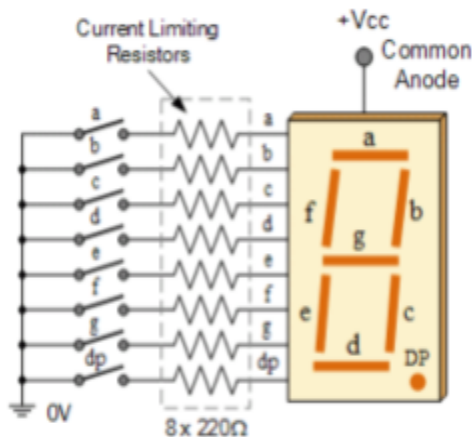


Figure 1. Pictorial representation of a seven segment display showing letter orientation.

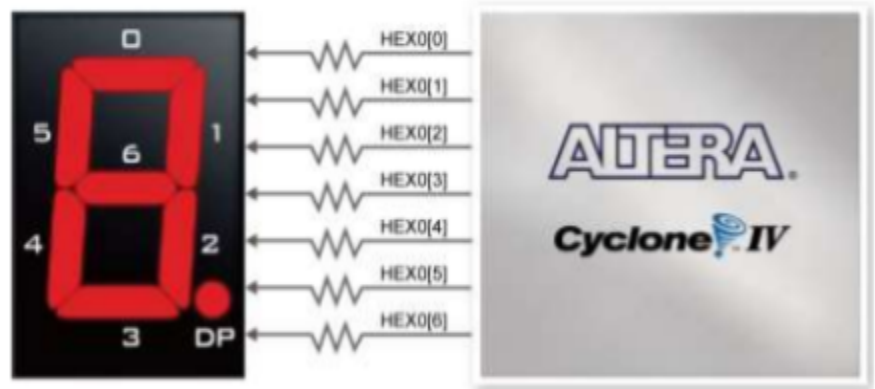


Figure 2. Pictorial representation of a seven segment display showing number orientation.

The rules of the game were based on a game called black jack. The game would be a card game that would

have to add up to 21. Originally this game is played with 52 cards. The goal of the game is to get 21 or close to it. The value of kings, queens, jacks are counted at 10. If an ace is picked up it can be held up as either a 1 or 11. When the cards are passed to each player, including the dealer each of them get only two cards transferred. The dealer has only one card facing down and the other players have their cards facing up. After that the dealer goes to each player and asks them if they want to pick up another card or stay with the cards they have. The players can have as many cards as they want if it does not go over the limit of 21. Additionally, if the dealer has 16 or under then they must pick up another card. Also, if the dealer has 21 the dealer wins, but if the dealer goes over 21 then all the other players win and then the deck of cards gets reshuffled.

But in the modified version, there would be two players playing. There would be three seven segment displays that would be controlled by each player. The random generator output number between one and nine. In total between the three players there would be six seven-segment-displays. The last two seven-segment displays controlled the score of each player. Another requirement is that each player has to pick up one card and that is the minimum and maximum. For example, if the score of player one is greater than player two then it will display 1 on the seven segment display for player one. Additionally if, For example, if the score of player two is greater than player one then it will display 2 on the seven segment display for player two. For one player the first two generated number would be added to make a total number A. Then there would be another randomly generated number B that would be added to A. Then the first two seven segment displays would give a total.

Design

Random Generator

The random generator has to output a random number, but in hardware that is not possible. So, a pseudorandom number has to be generated. Meaning that it is a sequence that is known. For a pseudo-random generator a linear feedback shift register has to be implemented where the last two bit has an XOR tied to it. In the modified black jack game there were only nine states. It could not be three bits since it could only go up to number 8. Since the state nine was needed, there had to be another bit. This gave a problem because since, there would be extra bits. Having four bits means that there are 16 states, but since there can be no state zero, there are 15 states. There can be no state zero, because it would stay in the same state forever because doing an xor with inputs 0 and 0 gives 0 and the number would not shift. The usable states that were needed were the number 1 to 9 and the other six states would have to be deleted, but that is impossible. So, the other states had to be equal to the other usable states. So states, 10, 11, 12, 13, 14, and 15 were unusable. What process that got implemented was that state 12 equal was made to state 2, state 11 was equal to state 3, state 10 was equal to state 1, state 13 was equal to state 9, state 14 was equal to state 5, and finally state 15 was equal to state 8. The linear feedback shift register is shown below:

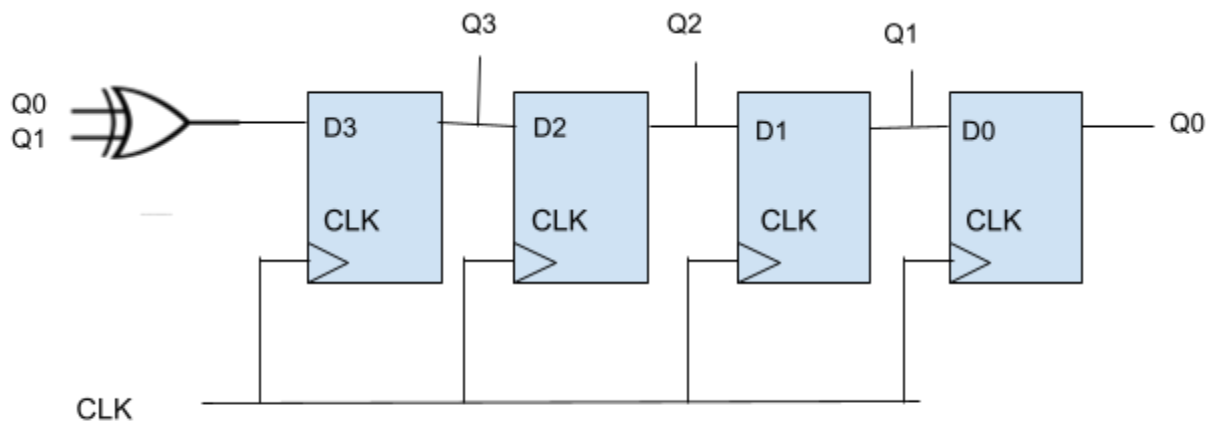


Fig.3 Linear feedback shift register that has four bits with the last two first two bits being in an XOR gate.

In the code there was a preset, that was controlled by a switch, since it needed to be started. In the code there is six random generator because six numbers need to have a different starting output sequence.

Seven-segment display

The seven-segment display in the modified black jack game was not how it was supposed to be in the de2-115 manual. The orientation was different. Instead of being a,b,c,d,e,f,g, the orientation that was g,f,e,d,c,b,a. Figure 4 below shows the display orientation:

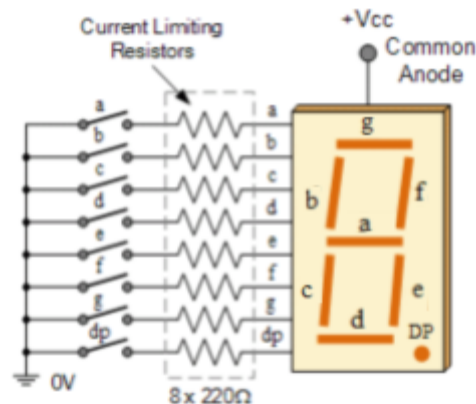


Figure 4. Pictorial representation of the modified letter orientation of the seven segment display.

The seven segment display had case statements that was used based on the four bit number of the random generator. It would use zero as input to turn on the segment and one as the input that turns off the segment. Below shows the truth table made in figure 5.

BCD	A	B	C	D	E	F	G	S
0000	1	0	0	0	0	0	0	0
0001	1	1	1	1	0	0	1	1
0010	0	1	0	0	1	0	0	2
0011	0	0	1	0	0	0	1	3
0100	0	0	1	1	0	0	1	4
0101	0	0	1	0	0	1	1	5
0110	0	0	0	0	0	1	0	6
0111	1	0	1	1	0	0	0	7
1000	0	0	0	0	0	0	0	8
1001	0	0	1	0	0	0	0	9

Figure 5. Truth table based on changed seven segment orientation.

This truth table shows the orientation of the case statements in the code. The seven segment display had to be controlled by either one of four switches to activate two numbers going one an added display. For example, two numbers such as 1 and 3 were being added, it had to display 04. That would do the process correctly but when adding a number such as 04 to 5, it could not do it, so switches being implemented made it easier to display the added number from the BCD adder.

Addition

The addition of two bcd numbers needed to be done and a bcd adder was used to do that. The bcd adder had a full adder and implemented four full adders in the bcd module. There was a separate module for doing the bcd addition and a separate module doing the addition of a full adder. The only thing that was wrong was that it would always subtract add one to the number that its supposed to be. So in the code it is subtracted by one.

Switches and physical setup

There were switches used to control the seven segment display. There are nine switches. The switches orientation can be seen below in the picture:

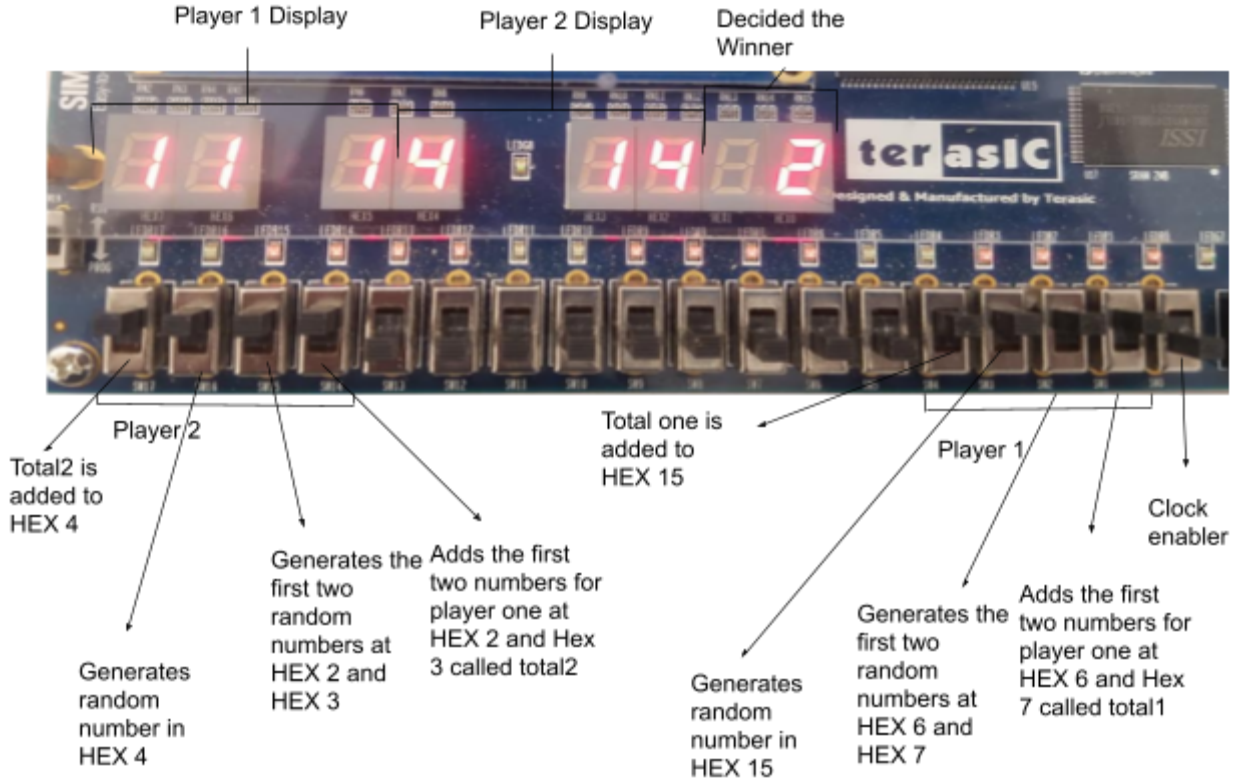


Figure 6. Shows switches orientation for each player and shows that player two has won.

Four switches controlled each player as it can be seen in figure 5. In the beginning to actually start the preset needs to be enabled, so that it can start the sequence from the random generator. There is no display since there is no switch turned random number generator switch turned on. The clock switch needs to be switch up and then down to enable the clock. Then either player can play. In the beginning of an example game the player one will have to turn on the switch in switch 2, so that is display the number. Keep in mind that the clock keeps running while the switch is off, which was a random sequence that can't be found since the display is off and the random generator runs, while the switch is turned off for all of the switches that output a random number. After the clock is enabled player one can play. Player one has to push up switch 2 to get two random numbers at HEX 7 and HEX 6. To add the two numbers together, the switch two has to be enabled. The added numbers will be called totalp1 to not get confused. After that the player will generate the third random number by enabling switch 15 to get a random number at HEX 5. To add totalp1 to hex 5, switch 4 is enabled. Additionally, after each switch is turned on, it should not be turned off because then it will not show the number. After that, the total number now is the final total for player one. Now it would be the time for player two to play. Player two also has four switch, and works almost the same way as player1. Player two has to turn on switch 15 to generate two random numbers at HEX 2 and HEX 3, which are going to be called totalp2. To add the two numbers switch 14 has to be enabled. To get the third random

number switch 16 had to be enabled and is displayed at HEX 4. To add totalp2 to HEX 4, switch 17 has to be enabled. Now the final total for player one is shown and the final total for player two is shown. If the final total for player one is less than the final total for player two then HEX 0 will display a two, indicating player two won. But, if player one was greater than player two, then player two wins. Figure five shows that player two one.

Results and Conclusion

Although the project did not simulate as close to black jack, it did use many functions together to work as one unit. The project had many errors that had to be fixed, but once it was fixed the game worked as indeed. The fgpa board correctly displays who wins, given a random number. This project included multiple parts, which were the random generator, the bcd adder, and the seven segment display. One feature that was missing because of time constraints was the LCD feature. It was supposed to display which player actually won. In the end there were many important concepts were learned. Additionally, the project was successfully completed and did what it was supposed to do.

References

- [1]J. Sachs, "Linear Feedback Shift Registers for the Uninitiated, Part I: Ex-Pralite Monks and Finite Fields - Jason Sachs", *Embeddedrelated.com*, 2019. [Online]. Available: <https://www.embeddedrelated.com/showarticle/1065.php>. [Accessed: 19- Dec- 2019]
- [2]"DE2-115 User Manual", *Intel.com*, 2019. [Online]. Available: https://www.intel.com/content/dam/altera-www/global/en_US/portal/dsn/42/doc-us-dsnbk-42-1404062209-de2-115-user-manual.pdf. [Accessed: 19- Dec- 2019]
- [3]"7-segment Display", *electronics*, 2019. [Online]. Available: <https://www.electronics-tutorials.ws/blog/7-segment-display-tutorial.html>. [Accessed: 19- Dec- 2019]
- [4]"Hit or Stand - Blackjack Rules and Strategy", *Hitorstand.net*, 2019. [Online]. Available: <http://www.hitorstand.net/strategy.php>. [Accessed: 19- Dec- 2019]

Appendices

Appendix A: System.v

// Main module used to connect other modules

```
module system(
```

```
input Clockd,
```

```
input Clockv,
```

```
input Presetv,
```

```
output reg[4:0]Qv,
```

```
output [3:0] pv,
```

```
output [6:0]segv,
```

```
output [6:0]segv1,
```

```
output [6:0]segv2,
```

```
output [3:0] pv2,
```

```
output [3:0] pv3,
```

```
output [6:0]segv3,
```

```
output [6:0]segv4,
```

```
output [6:0]segv5,
```

```
output [6:0]segv6,
```

```
output [6:0]Hex1,
```

```
output [6:0]Hex2,
```

```
input cin8_inv,
```

```
output [11:0] sum12_ouvt,
```

```
output [7:0] sum8_outv,
```

```
output [11:0] sum12_ouvtv2,
```

```
output [11:0] sum12_ouvtv5,
```

```
output [7:0] sum8_ouvtv5,
```

```
output [7:0] sum8_outv2,
```

```
output cout8_outv,
```

```
output [3:0]av,
```

```
output [3:0]bv,
```

```
input addv,
```

```
input switch,
```

```
input switch2,
```

```

input switch3,
input sw4,
input sw5,
////////////////
output [6:0]segvv2,
output [6:0]segv1v2,
output [6:0]segv2v2,
output [3:0] pv2v2,
output [3:0] pv3v2,
output [3:0] pvv2,
output [6:0]segv3v2,
output [6:0]segv4v2,

output [6:0]segv5v2,
output [6:0]segv6v2,

```

```

input cin8_invv2,
output [11:0] sum12_ouvtvv2,
output [7:0] sum8_outvv2,
output [11:0] sum12_ouvtv2v2,
output [11:0] sum12_ouvtv5v2,
output [7:0] sum8_ouvtv5v2,
output [7:0] sum8_outv2v2,
output cout8_outvv2,
output [3:0]avv2,
output [3:0]bv2,
input addvv2,
input switchv2,
input switch2v2,
input switch3v2,
input sw4v2,
input sw5v2

```

```

);

```

```

wire [3:0] ab;
wire [7:0] bc;

```

```
Binary2BCD (  
  .bin(pv),  
  .bcd(bc)  
);
```

```
  wire [3:0] ab2;  
  wire [7:0] bc2;
```

```
  random random1(  
    .switch(switch),  
    .CLOCK_50(cout),  
    .Preset(Presetv),  
    .p(pv)  
  );
```

```
  random2 random2(  
    .switch(switch),  
    .CLOCK_50(coutd),  
    .Preset(Presetv),  
    .p(pv2)  
  );
```

```
  random3 random3(  
    .switch(switch3),  
    .CLOCK_50(cout),  
    .Preset(Presetv),  
    .p(pv3)  
  );
```

```
Binary2BCD (  
  .bin(pv2),  
  .bcd(bc2)
```

```
);
wire [3:0] ab3;
wire [7:0] bc3;
```

```
Binary2BCD (
  .bin(pv3),
  .bcd(bc3)
);
```

```
bcd_to_seven_seg hex5(
  .bcd(bc3),
  .segv(segv3),
  .switch(switch3)
);
```

```
//////////
bcd_to_seven_seg2 hex6(
  .bcd(bc),
  // .seg(segv),
  .bcd2(av2),//
  .bcd3(av3),///
  .bcd4(av4),
  .segv(segv),
  // .segv2(),//
  .switch(switch),
  .switch2(addv),
  .switch3(sw4),///
  .switch4(sw5)
);
```

```
////////////////////////////////////
bcd_to_seven_seg2 hex7(
  .bcd(bc2),
  // .seg(segv),
  .bcd2(bv2),//
  .bcd3(bv3),///
  .bcd4(bv4),
  .segv(segv2),
  // .segv2(),//
  .switch(switch),
  .switch2(addv),
```

```
.switch3(sw4),///
.switch4(sw5)
);
//////////
//start of next segments for player 2
```

```
wire [3:0] av2;
wire [3:0] bv2;
```

```
wire [3:0] av3;
wire [3:0] bv3;
```

```
wire [3:0] av4;
wire [3:0] bv4;
```

```
wire [7:0] sum8_outv3;
wire [7:0] sum8_outv4;
```

```
bcdAdder(
    .a8_in(bc),
    .b8_in(bc2),
    .cin8_in(cin8_inv),
    .sum12_out(sum12_ouvt),
    .cout8_out(cout8_outv),
    .a(av2),
    .b(bv2),
    .add(addv),
    //switch()
    .sum8_out(sum8_outv3)
);
```

```
wire [7:0] sum8_outv5;
assign sum8_outv4=sum8_outv3;
```

```
bcdAdder2 add(
    .a8_in(sum8_outv3),
    .b8_in(bc3),
    .cin8_in(cin8_inv),
    .sum12_out(sum12_ouvtv2),
    .cout8_out(sum8_ouvtv2),
```

```

        .a(av3),
        .b(bv3),
        .add(switch3),
        .sum8_out(sum8_outv5)
    );

```

```

////////////////////////////////

```

```

wire [3:0] abv2;
wire [7:0] bcv2;

```

```

    wire [3:0] ab2v2;
    wire [7:0] bc2v2;

```

```

    wire [3:0] av2v2;
    wire [3:0] bv2v2;

```

```

    wire [3:0] av3v2;
    wire [3:0] bv3v2;

```

```

    wire [3:0] av4v2;
    wire [3:0] bv4v2;

```

```

    wire [7:0] bc3v2;
    wire [7:0] sum8_outv3v2;

```

```

        wire [3:0] ab3v2;

```

```

random6 (
    .switch(switchv2),
    .CLOCK_50(cout),
    .Preset(Presetv),
    .p(pvv2)
);

```

```

random2 random2v2(
    .switch(switchv2),
    .CLOCK_50(coutd),

```

```
.Preset(Presetv),
.p(pv2v2)
);
```

```
random3 random3v2(
.switch(switch3v2),
.CLOCK_50(cout),
.Preset(Presetv),
.p(pv3v2)
);
```

```
Binary2BCD (
.bin(pvv2),
.bcd(bcv2)
);
```

```
Binary2BCD (
.bin(pv2v2),
.bcd(bc2v2)
);
```

```
Binary2BCD (
.bin(pv3v2),
.bcd(bc3v2)
);
```

```
bcd_to_seven_seg hex4(
.bcd(bc3v2),
//.seg(segv),
.segv(segv3v2),
.switch(switch3v2)
);
```

```
bcd_to_seven_seg2 hex3(
.bcd(bcv2),
//.seg(segv),
.bcd2(av2v2),//
.bcd3(av3v2),//
.bcd4(av4v2),
```

```
.segv(segvv2),
```

```
.switch(switchv2),
.switch2(addvv2),
.switch3(sw4v2),///
.switch4(sw5v2)
);
```

```
bcd_to_seven_seg2 hex2(
.bcd(bc2v2),
```

```
.bcd2(bv2v2),//
.bcd3(bv3v2),///
.bcd4(bv4v2),
.segv(segv2v2),
```

```
.switch(switchv2),
.switch2(addvv2),
.switch3(sw4v2),///
.switch4(sw5v2)
);
```

```
bcdAdder(
    .a8_in(bcv2),
    .b8_in(bc2v2),
    .cin8_in(cin8_invv2),
    .sum12_out(sum12_ouvtvv2),
    .cout8_out(cout8_outvv2),
    .a(av2v2),
    .b(bv2v2),
    .add(addvv2),

    .sum8_out(sum8_outv3v2)
);
```

```
bcdAdder2 addv2(
    .a8_in(sum8_outv3v2),
    .b8_in(bc3v2),
    .cin8_in(cin8_invv2),
```

```

        .sum12_out(sum12_ouvtv2v2),
        .cout8_out(sum8_ouvtv2v2),
        .a(av3v2),
        .b(bv3v2),
        .add(switch3v2),
        .sum8_out(sum8_outv5v2)
    );

```

```

wire [3:0]bin;
assign bin=4'b1100;
wire [3:0]bin2;
assign bin2=4'b10011;
wire [7:0]bina;
wire [7:0]binb;
//////////

```

```

Binary2BCD (
    .bin(bin),
    .bcd(bina)
);
Binary2BCD (
    .bin(bin2),
    .bcd(binb)
);
wire [7:0] cin8_invv8;
wire [7:0] sum8_outv8;
wire [11:0]sum12_ouvtvv8;
wire [7:0] cout8_outvv8;
wire [3:0]av2v8;
wire [3:0]bv2v8;
bcdAdder(
    .a8_in(bina),
    .b8_in(binb),
    .cin8_in(cin8_invv8),
    .sum12_out(sum12_ouvtvv8),
    .cout8_out(cout8_outvv8),
    .a(av2v8),
    .b(bv2v8),
    .add(addvv8),
    //switch()
    .sum8_out(sum8_outv8)
);

```

```

always @(posedge Clockv)//&&(sum12_ouvtv2<sum12_ouvtv2v2)
)||((sum12_ouvtv2>sum12_ouvtv2v2) //&&(sum12_ouvtv2>sum12_ouvtv2v2)
)||((sum12_ouvtv2v2>sum12_ouvtv2)

```

```

begin
    if((sum12_ouvtv2>sum12_ouvtv2v2) && ((addv==1)&& (switch==1)&& (switch3==1)&&
(sw4==1) && (addvv2==1)&& (switchv2==1)&& (switch3v2==1)&& (sw4v2==1)))
        begin

```

```

            Hex2=7'b1111111;
            Hex1=7'b1111001;

```

```

        end
        else if((sum12_ouvtv2v2>sum12_ouvtv2) && ((addv==1)&& (switch==1)&&
(switch3==1)&& (sw4==1) && (addvv2==1)&& (switchv2==1)&& (switch3v2==1)&&
(sw4v2==1)))
            begin

```

```

                Hex1=7'b1111111;
                Hex2=7'b0100100;
            end

```

```

        else
        begin
            Hex2=7'b1111111;
            Hex1=7'b1111111;
        end

```

```

end

```

```

        reg [24:0] counter;
        reg [24:0] counterd;
        reg cout;
        reg coutd;
    initial begin

```

```

        counter=0;
        cout=0;

```

```

end
//Clock slowed down
always @(posedge Clockv)

begin
    if(counter==0)
        begin counter<=24999999;
            cout<=~cout;

            end
        else
            begin counter<=counter-1;
                end
    end

always @(posedge Clockd)

begin
    if(counterd==0)
        begin counterd<=(24999999/2);
            coutd<=~coutd;
            end
        else
            begin counterd<=counterd-1;
                end
    end

end

endmodule

```

Appendix B: bcdAdder2.v

```

module bcdAdder2(a8_in, b8_in, sum12_out, cout8_out, cin8_in,a,b,add,sum8_out,add2);
    input [7:0] a8_in, b8_in;
    input cin8_in;
    output [11:0] sum12_out;
    output [7:0] sum8_out;
    output cout8_out;
    output reg[3:0]a;
    output reg[3:0]b;
    wire cout8_out,cin8_in;

```

```

wire [7:0] a8_in, b8_in;
wire [11:0] sum12_out;
wire [7:0] sum8_out;
wire [7:0] sumtemp;
wire [3:0] couttemp, fba2_b4, fba3_b4;
input add2;

input add;

fourBitAdder fba0(
    .a4_in(a8_in[3:0]),
    .b4_in(b8_in[3:0]),
    .sum4_out(sumtemp[3:0]),
    .cin4_in(cin8_in),
    .cout4_out(couttemp[0])
);

assign couttemp[1] = (sumtemp[3]&sumtemp[2])|(sumtemp[3]&sumtemp[1])|couttemp[0];

fourBitAdder fba1(
    .a4_in(a8_in[7:4]),
    .b4_in(b8_in[7:4]),
    .sum4_out(sumtemp[7:4]),
    .cin4_in(couttemp[1]),
    .cout4_out(couttemp[2])
);

assign couttemp[3] = (sumtemp[7]&sumtemp[6])|(sumtemp[7]&sumtemp[5])|couttemp[2];
assign fba2_b4[1] = couttemp[1];
assign fba2_b4[2] = couttemp[1];
assign fba2_b4[0] = 1'b0;
assign fba2_b4[3] = 1'b0;

fourBitAdder fba2(
    .a4_in(sumtemp[3:0]),
    .b4_in(fba2_b4[3:0]),
    .sum4_out(sum12_out[3:0]),
    .cin4_in(1'b0),
    .cout4_out()
);

assign fba3_b4[1] = couttemp[3];
assign fba3_b4[2] = couttemp[3];
assign fba3_b4[0] = 1'b0;
assign fba3_b4[3] = 1'b0;

```

```

fourBitAdder fba3(
    .a4_in(sumtemp[7:4]),
    .b4_in(fba3_b4[3:0]),
    .sum4_out(sum12_out[7:4]),
    .cin4_in(1'b0),
    .cout4_out()
);
assign sum12_out[11:9] = 3'b000;
assign sum12_out[8] = couttemp[3];
assign sum8_out[7:0] = sum12_out[7:0] - 7'b00000001;
always @(posedge add)
begin
if(add==1)

a[3:0] <= sum8_out[3:0];
b[3:0] <= sum8_out[7:4];

end

endmodule

```

Appendix C: Binary2BCD.v

//Code given by Professor

```
module Binary2BCD (bin, bcd);
```

```
input [4:0] bin;
```

```
output [7:0] bcd;
```

```
reg [7:0] bcd;
```

```
reg [3:0] i;
```

```
always @(bin)
```

```
begin
```

```
bcd = 0;
```

```
for (i = 0; i < 5; i = i + 1) //repeat 5 times since binary has 5 bits
```

```
begin
```

```
bcd = {bcd[6:0], bin[4-i]}; //shift bcd number left once, with LSB from input binary #
```

```
if (i < 4 && bcd[3:0] > 4 ) bcd [3:0] = bcd[3:0] + 3; // add 3 to hex digit if it is more than 4.
```

```

if (i < 4 && bcd[7:4] > 4 ) bcd [7:4] = bcd[7:4] + 3;
if(i < 4 && bcd[7:5] > 4) bcd[7:5] = bcd[7:5] + 3;
end
end
endmodule

```

Appendix D: bcdAdder.v

```

module bcdAdder(a8_in, b8_in, sum12_out, cout8_out, cin8_in,a,b,add,sum8_out);
//Declarations of inputs,outputs, and wires
    input [7:0] a8_in, b8_in;
    input cin8_in;
    output [11:0] sum12_out;
    output [7:0] sum8_out;
    output cout8_out;
    output reg[3:0]a;
    output reg[3:0]b;
    wire cout8_out,cin8_in;
    wire [7:0] a8_in, b8_in;
    wire [11:0] sum12_out;
    wire [7:0] sum8_out;
    wire [7:0] sumtemp;
    wire [3:0] couttemp, fba2_b4, fba3_b4;

    input add;
// 4 Fourbit adder used as modules
    fourBitAdder fba0(
        .a4_in(a8_in[3:0]),
        .b4_in(b8_in[3:0]),
        .sum4_out(sumtemp[3:0]),
        .cin4_in(cin8_in),
        .cout4_out(couttemp[0])
    );
    assign couttemp[1] = (sumtemp[3]&sumtemp[2])|(sumtemp[3]&sumtemp[1])|couttemp[0];

    fourBitAdder fba1(
        .a4_in(a8_in[7:4]),
        .b4_in(b8_in[7:4]),
        .sum4_out(sumtemp[7:4]),
        .cin4_in(couttemp[1]),
        .cout4_out(couttemp[2])
    );
    assign couttemp[3] = (sumtemp[7]&sumtemp[6])|(sumtemp[7]&sumtemp[5])|couttemp[2];

```

```

assign fba2_b4[1] = couttemp[1];
assign fba2_b4[2] = couttemp[1];
assign fba2_b4[0] = 1'b0;
assign fba2_b4[3] = 1'b0;

fourBitAdder fba2(
    .a4_in(sumtemp[3:0]),
    .b4_in(fba2_b4[3:0]),
    .sum4_out(sum12_out[3:0]),
    .cin4_in(1'b0),
    .cout4_out()
);
assign fba3_b4[1] = couttemp[3];
assign fba3_b4[2] = couttemp[3];
assign fba3_b4[0] = 1'b0;
assign fba3_b4[3] = 1'b0;

fourBitAdder fba3(
    .a4_in(sumtemp[7:4]),
    .b4_in(fba3_b4[3:0]),
    .sum4_out(sum12_out[7:4]),
    .cin4_in(1'b0),
    .cout4_out()
);
assign sum12_out[11:9] = 3'b000;
assign sum12_out[8] = couttemp[3];

assign sum8_out[7:0] = sum12_out[7:0] - 7'b00000001;

always @(posedge add)
begin

a[3:0] <= sum8_out[3:0];
b[3:0] <= sum8_out[7:4];

end

endmodule

```

Appendix E: random6.v

```

module random6(switch,CLOCK_50, Preset,p);
//declaration of inputs and outputs
input Preset;
input CLOCK_50;
reg[3:0]Q;
reg[3:0]y;
//P vector used as output to test waveforms in quartus
//Also used to test LEDs when seven segment display was not implemented yet
output reg[3:0]p;
input switch;

always
begin
//Start the sequence when preset equals 1
//random generator start at sequence 4'b0111, because it needs to start other than state zero.
if(Preset==1) y<=4'b0111;
//if switch equals zero continue sequence other wise stop if switch is one
else if(switch==0)
begin

begin
//Linear feed back shift register with first two bit having an XOR

y[3] = (Q[1]^Q[0]);
y[2] = Q[3];
y[1] = Q[2];
y[0] = Q[1];

//State at state less than 2'd10

if(y[3:0]<4'b1010)
begin
p<=y;
end
//State 4'b01100 is now equal to state 4'b0010
else if(y[3:0]==4'b01100)
begin

```

```

        p<=4'b0010;
    end
//State 4'b1011 is now equal to state 4'b0011
    else if(y[3:0]==4'b1011)
    begin
        p<=4'b0011;
    end
//State 4'b1010 is now equal to state 4'b0001

    else if(y[3:0]==4'b1010)
    begin
        p<=4'b0001;
    end
//State 4'b1101 is now equal to state 4'b1001

    else if(y[3:0]==4'b1101)
    begin
        p<=4'b1001;
    end
//State 4'b1110 is now equal to state 4'b0101

    else if(y[3:0]==4'b1110)
    begin
        p<=4'b0101;
    end
//State 4'b1111 is now equal to state 4'b1000

    else if(y[3:0]==4'b1111)
    begin
        p<=4'b1000;
    end
    end

end

end

//Starts at positive edge of clock
// Q vector is equal to y vector
//p vector is equal to itslef always at the positive edge

```

```
// equal to it self to start
```

```
always @(posedge CLOCK_50)
begin
```

```
Q[3:0]<=y[3:0];
p[3:0]<=p[3:0];
```

```
end
```

```
endmodule
```

Appendix F: bcd to seven_seg2.v

//BCD to seven-segment-display based on three random three seven segment displays

```
module bcd_to_seven_seg2(bcd,bcd2,bcd3,bcd4,segv,switch,switch2,switch3,switch4);
```

```
input [3:0]bcd;
input [3:0]bcd2;
input [3:0]bcd3;
input [3:0]bcd4;
reg[6:0]seg;
output reg[6:0]segv;
input switch;
input switch2;
input switch3;
input switch4;
```

```
always @(*)
begin
if(switch4==1)
begin
```

```
    case(bcd4)
    0:seg=7'b1000000;
    1:seg=7'b1111001;
    2:seg=7'b0100100;
    3:segv=7'b0110000;
    4:seg=7'b0011001;
    5:seg=7'b0010010;
    6:seg=7'b0000010;
```

```

7:seg=7'b1011000;
8:seg=7'b0000000;
9:seg=7'b0010000;
endcase

```

```

if(switch==1)
begin
    segv=seg;

```

```

end

```

```

else if(switch2==0 || switch3==0 ||switch==0 )////////
begin

```

```

    segv=7'b1111111;
end

```

```

end

```

```

////////////////////////////////////////

```

```

if(switch3==1)

```

```

begin

```

```

    case(bcd3)
    0:seg=7'b1000000;
    1:seg=7'b1111001;
    2:seg=7'b0100100;
    3:segv=7'b0110000;
    4:seg=7'b0011001;
    5:seg=7'b0010010;
    6:seg=7'b0000010;
    7:seg=7'b1011000;
    8:seg=7'b0000000;
    9:seg=7'b0010000;
    endcase

```

```

if(switch==1)
begin
    segv=seg;

```

```

end

```

```

else if(switch2==0 || switch3==0 ||switch==0 )////////
begin

    segv=7'b1111111;
end

end

//////////

else if(switch2==1)

begin

    case(bcd2)
    0:seg=7'b1000000;
    1:seg=7'b1111001;
    2:seg=7'b0100100;
    3:seg=7'b0110000;
    4:seg=7'b0011001;
    5:seg=7'b0010010;
    6:seg=7'b0000010;
    7:seg=7'b1011000;
    8:seg=7'b0000000;
    9:seg=7'b0010000;
    endcase

    if(switch==1)
    begin
        segv=seg;

    end

else if(switch2==0 && switch3==0)
begin

    segv=7'b1111111;

end

```

end

else

/*

begin

//seg=7'b1111111;

segv=7'b1111111;

end

*/

begin

case(bcd)

0:seg=7'b1000000;

1:seg=7'b1111001;

2:seg=7'b0100100;

3:seg=7'b0110000;

4:seg=7'b0011001;

5:seg=7'b0010010;

6:seg=7'b0000010;

7:seg=7'b1011000;

8:seg=7'b0000000;

9:seg=7'b0010000;

endcase

if(switch==1)

begin

segv=seg;

//break;

end

else

begin

//seg=7'b1111111;

segv=7'b1111111;

end

end

end
endmodule

Appendix G: bcd_to_seven_seg.v

//BCD to seven segment display based on one random number

module bcd_to_seven_seg(bcd,segv,switch);

//Declarations of inputs and outputs

input [3:0]bcd;

reg[6:0]seg;

output reg[6:0]segv;

input switch;

always@(*)

begin

//Based on bcd number it would display the corresponding number

//on the seven-segment-display

case(bcd)

0:seg=7'b1000000;

1:seg=7'b1111001;

2:seg=7'b0100100;

3:seg=7'b0110000;

4:seg=7'b0011001;

5:seg=7'b0010010;

6:seg=7'b0000010;

7:seg=7'b1011000;

```

        8:seg=7'b0000000;
        9:seg=7'b0010000;
    endcase
// Seven-segment display would display a one if the switch is on
// Otherwise the seven-segment display would not light up
if(switch==1)
begin
    segv=seg;
    //break;
end
else
begin

segv=7'b1111111;
end
end

endmodule

```

Appendix H: random2

```

module random2(switch,CLOCK_50, Preset,p);
//declaration of inputs and outputs
input Preset;
input CLOCK_50;
reg[3:0]Q;
reg[3:0]y;
//P vector used as output to test waveforms in quartus
//Also used to test LEDs when seven segment display was not implemented yet
output reg[3:0]p;
input switch;

always
begin
//Start the sequence when preset equals 1
//random generator start at sequence 4'b1001, because it needs to start other than state zero.
if(Preset==1) y<=4'b1001;
//if switch equals zero continue sequence other wise stop if switch is one
else if(switch==0)
begin

```

```

begin
//Linear feed back shift register with first two bit having an XOR

y[3] = (Q[1]^Q[0]);
y[2] = Q[3];
y[1] = Q[2];
y[0] = Q[1];

//Stats at state less then 2'd10

        if(y[3:0]<4'b1010)
        begin
        p<=y;
        end
//State 4'b01100 is now equal to state 4'b0010
        else if(y[3:0]==4'b01100)
        begin
        p<=4'b0010;
        end
//State 4'b1011 is now equal to state 4'b0011
        else if(y[3:0]==4'b1011)
        begin
        p<=4'b0011;
        end
//State 4'b1010 is now equal to state 4'b0001

        else if(y[3:0]==4'b1010)
        begin
        p<=4'b0001;
        end
//State 4'b1101 is now equal to state 4'b1001

        else if(y[3:0]==4'b1101)
        begin
        p<=4'b1001;
        end
//State 4'b1110 is now equal to state 4'b0101

        else if(y[3:0]==4'b1110)
        begin

```

```

        p<=4'b0101;
    end
//State 4'b1111 is now equal to state 4'b1000

        else if(y[3:0]==4'b1111)
        begin
            p<=4'b1000;
        end
    end

end

end

end

//Starts at positive edge of clock
// Q vector is equal to y vector
//p vector is equal to itslef always at the positive edge
// equal to it self to start

always @(posedge CLOCK_50)
begin

    Q[3:0]<=y[3:0];
    p[3:0]<=p[3:0];

end

endmodule

```

Appendix I: random3

```

module random3(switch,CLOCK_50, Preset,p);
//declaration of inputs and outputs
input Preset;
input CLOCK_50;
reg[3:0]Q;
reg[3:0]y;

```

```

//P vector used as output to test waveforms in quartus
//Also used to test LEDs when seven segment display was not implemented yet
output reg[3:0]p;
input switch;

always
begin
//Start the sequence when preset equals 1
//random generator start at sequence 4'b0110, because it needs to start other than state zero.
if(Preset==1) y<=4'b0110;
//if switch equals zero continue sequence other wise stop if switch is one
else if(switch==0)
begin

begin
//Linear feed back shift register with first two bit having an XOR

y[3] = (Q[1]^Q[0]);
y[2] = Q[3];
y[1] = Q[2];
y[0] = Q[1];

//State at state less than 2'd10

if(y[3:0]<4'b1010)
begin
p<=y;
end
//State 4'b01100 is now equal to state 4'b0010
else if(y[3:0]==4'b01100)
begin
p<=4'b0010;
end
//State 4'b1011 is now equal to state 4'b0011
else if(y[3:0]==4'b1011)
begin
p<=4'b0011;
end
//State 4'b1010 is now equal to state 4'b0001

```

```

        else if(y[3:0]==4'b1010)
        begin
            p<=4'b0001;
        end
//State 4'b1101 is now equal to state 4'b1001

        else if(y[3:0]==4'b1101)
        begin
            p<=4'b1001;
        end
//State 4'b1110 is now equal to state 4'b0101

        else if(y[3:0]==4'b1110)
        begin
            p<=4'b0101;
        end
//State 4'b1111 is now equal to state 4'b1000

        else if(y[3:0]==4'b1111)
        begin
            p<=4'b1000;
        end
    end

end

end

//Starts at positive edge of clock
// Q vector is equal to y vector
//p vector is equal to itself always at the positive edge
// equal to it self to start

always @(posedge CLOCK_50)
begin

Q[3:0]<=y[3:0];
p[3:0]<=p[3:0];

```

end

endmodule

Appendix I: random2

```

module random(switch,CLOCK_50, Preset,p);
//declaration of inputs and outputs
input Preset;
input CLOCK_50;
reg[3:0]Q;
reg[3:0]y;
output reg[3:0]p;
input switch;

always
begin
//Start the sequence when preset equals 1
//random generator start at sequence 4'b0111, because it needs to start other than state zero.
if(Preset==1) y<=4'b1000;
//if switch equals zero continue sequence other wise stop if switch is one
else if(switch==0)
begin

begin
//Linear feed back shift register with first two bit having an XOR

y[3] = (Q[1]^Q[0]);
y[2] = Q[3];
y[1] = Q[2];
y[0] = Q[1];

//Stats at state less than 2'd10

if(y[3:0]<4'b1010)
begin
p<=y;
end

```

```
//State 4'b01100 is now equal to state 4'b0010
    else if(y[3:0]==4'b01100)
    begin
        p<=4'b0010;
    end
//State 4'b1011 is now equal to state 4'b0011
    else if(y[3:0]==4'b1011)
    begin
        p<=4'b0011;
    end
//State 4'b1010 is now equal to state 4'b0001

    else if(y[3:0]==4'b1010)
    begin
        p<=4'b0001;
    end
//State 4'b1101 is now equal to state 4'b1001

    else if(y[3:0]==4'b1101)
    begin
        p<=4'b1001;
    end
//State 4'b1110 is now equal to state 4'b0101

    else if(y[3:0]==4'b1110)
    begin
        p<=4'b0101;
    end
//State 4'b1111 is now equal to state 4'b1000

    else if(y[3:0]==4'b1111)
    begin
        p<=4'b1000;
    end
    end

end

end
```

```
//Starts at positive edge of clock  
// Q vector is equal to y vector  
//p vector is equal to itself always at the positive edge  
// equal to it self to start
```

```
always @(posedge CLOCK_50)  
begin
```

```
Q[3:0]<=y[3:0];  
p[3:0]<=p[3:0];
```

```
end
```

```
endmodule
```

Appendix J: Pin Assignments

	itatu:	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1			Clockd	Location	PIN_AG14	Yes			
2			Clockv	Location	PIN_Y2	Yes			
3			Presetv	Location	PIN_AB28	Yes			
4			addv	Location	PIN_AC28	Yes			
5			segv[6]	Location	PIN_AC17	Yes			
6			segv[5]	Location	PIN_AA15	Yes			
7			segv[4]	Location	PIN_AB15	Yes			
8			segv[3]	Location	PIN_AB17	Yes			
9			segv[2]	Location	PIN_AA16	Yes			
10			segv[1]	Location	PIN_AB16	Yes			
11			segv[0]	Location	PIN_AA17	Yes			
12			segv2[6]	Location	PIN_AA14	Yes			
13			segv2[5]	Location	PIN_AG18	Yes			
14			segv2[4]	Location	PIN_AF17	Yes			
15			segv2[3]	Location	PIN_AH17	Yes			
16			segv2[2]	Location	PIN_AG17	Yes			
17			segv2[1]	Location	PIN_AE17	Yes			
18			segv2[0]	Location	PIN_AD17	Yes			
19			segv3[6]	Location	PIN_AH18	Yes			
20			segv3[5]	Location	PIN_AF18	Yes			
21			segv3[4]	Location	PIN_AG19	Yes			
22			segv3[3]	Location	PIN_AH19	Yes			
23			segv3[2]	Location	PIN_AB18	Yes			
24			segv3[1]	Location	PIN_AC18	Yes			
25			segv3[0]	Location	PIN_AD18	Yes			
26			sw4	Location	PIN_AB27	Yes			
27			switch	Location	PIN_AC27	Yes			
28			switch3	Location	PIN_AD27	Yes			
29			addvv2	Location	PIN_AA23	Yes			
30			switchv2	Location	PIN_AA22	Yes			
31			switch3v2	Location	PIN_Y24	Yes			
32			sw4v2	Location	PIN_Y23	Yes			
33			segvv2[0]	Location	PIN_AA25	Yes			
34			segvv2[1]	Location	PIN_AA26	Yes			
35			segvv2[2]	Location	PIN_Y25	Yes			
36			segvv2[3]	Location	PIN_W26	Yes			
37			segvv2[4]	Location	PIN_Y26	Yes			
38			segvv2[5]	Location	PIN_W27	Yes			
39			segvv2[6]	Location	PIN_W28	Yes			
40			segv2v2[0]	Location	PIN_V21	Yes			
41			segv2v2[1]	Location	PIN_U21	Yes			

42	✓		 segv2v2[2]	Location	PIN_AB20	Yes			
43	✓		 segv2v2[3]	Location	PIN_AA21	Yes			
44	✓		 segv2v2[4]	Location	PIN_AD24	Yes			
45	✓		 segv2v2[5]	Location	PIN_AF23	Yes			
46	✓		 segv2v2[6]	Location	PIN_Y19	Yes			
47	✓		 segv3v2[0]	Location	PIN_AB19	Yes			
48	✓		 segv3v2[1]	Location	PIN_AA19	Yes			
49	✓		 segv3v2[2]	Location	PIN_AG21	Yes			
50	✓		 segv3v2[3]	Location	PIN_AH21	Yes			
51	✓		 segv3v2[4]	Location	PIN_AE19	Yes			
52	✓		 segv3v2[5]	Location	PIN_AF19	Yes			
53	✓		 segv3v2[6]	Location	PIN_AE18	Yes			
54	✓		 Hex1[6]	Location	PIN_U24	Yes			
55	✓		 Hex1[5]	Location	PIN_U23	Yes			
56	✓		 Hex1[4]	Location	PIN_W25	Yes			
57	✓		 Hex1[3]	Location	PIN_W22	Yes			
58	✓		 Hex1[2]	Location	PIN_W21	Yes			
59	✓		 Hex1[1]	Location	PIN_Y22	Yes			
60	✓		 Hex1[0]	Location	PIN_M24	Yes			
61	✓		 Hex2[6]	Location	PIN_H22	Yes			
62	✓		 Hex2[5]	Location	PIN_J22	Yes			
63	✓		 Hex2[4]	Location	PIN_L25	Yes			
64	✓		 Hex2[3]	Location	PIN_L26	Yes			
65	✓		 Hex2[2]	Location	PIN_E17	Yes			
66	✓		 Hex2[1]	Location	PIN_F22	Yes			
67	✓		 Hex2[0]	Location	PIN_G18	Yes			